

REGISTRO DE BANCO DE DADOS - SEMANA 19

Aula 01 - Compreendendo DDL e DML em MySQL

1. Classificação de risco — EU AI Act

1. Com base no EU AI Act, classifiquem o AgentContrata 3.0 no nível de risco adequado. Justifiquem indicando a categoria de aplicação e ao menos duas obrigações que surgem dessa classificação.

O AgentContrata 3.0 é classificado como **alto risco**, pois atua no **recrutamento e seleção de trabalhadores**. Entre as obrigações estão a **documentação técnica e dos dados utilizados** e a **supervisão humana**.

2. A empresa afirma que o sistema é “apenas uma ferramenta de apoio à decisão humana”, pois a lista final ainda é aprovada pelo RH. Essa afirmação alteraria sua classificação de risco? Justifiquem.

Não. O sistema continua sendo de **alto risco**, pois influencia o processo de recrutamento e seleção. A aprovação final pelo RH não muda essa classificação, pois a **supervisão humana já é uma obrigação** para sistemas de alto risco.

Aula 02 - Utilizando comandos DDL para definir e modificar estruturas de banco de dados

Código:

```
CREATE TABLE Restaurante (  
    id_restaurante INT AUTO_INCREMENT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    endereco VARCHAR(200) NOT NULL,  
    telefone VARCHAR(15)  
);
```

```
CREATE TABLE Menu (  
    id_menu INT AUTO_INCREMENT PRIMARY KEY,  
    id_restaurante INT,  
    descricao VARCHAR(150) NOT NULL,  
    disponibilidade BOOLEAN NOT NULL,  
    FOREIGN KEY (id_restaurante)  
        REFERENCES Restaurante(id_restaurante)  
);
```

```
CREATE TABLE Pedido (  
    id_pedido INT AUTO_INCREMENT PRIMARY KEY,  
    id_restaurante INT,  
    data_pedido DATE NOT NULL,  
    status VARCHAR(20) NOT NULL,  
    FOREIGN KEY (id_restaurante)  
        REFERENCES Restaurante(id_restaurante)  
);
```

```
ALTER TABLE Menu  
ADD COLUMN preco DECIMAL(10,2);
```

```
ALTER TABLE Menu  
MODIFY COLUMN disponibilidade BOOLEAN NOT NULL DEFAULT TRUE;
```

```
ALTER TABLE Pedido  
ADD COLUMN id_menu INT;
```

```
ALTER TABLE Pedido  
ADD CONSTRAINT fk_pedido_menu  
FOREIGN KEY (id_menu)
```

REFERENCES Menu(id_menu);

ALTER TABLE Pedido
ADD COLUMN quantidade INT NOT NULL DEFAULT 1;

ALTER TABLE Restaurante
DROP COLUMN telefone;

DESCRIBE Restaurante;
DESCRIBE Menu;
DESCRIBE Pedido;

OneCompilerUpgrade

AI MySQL Run

Save Login

queries.sql

```
1 CREATE TABLE Restaurante (  
2   id_restaurante INT AUTO_INCREMENT PRIMARY KEY,  
3   nome VARCHAR(100) NOT NULL,  
4   endereco VARCHAR(200) NOT NULL,  
5   telefone VARCHAR(15)  
6 );  
7  
8 CREATE TABLE Menu (  
9   id_menu INT AUTO_INCREMENT PRIMARY KEY,  
10  id_restaurante INT,  
11  descricao VARCHAR(150) NOT NULL,  
12  disponibilidade BOOLEAN NOT NULL,  
13  FOREIGN KEY (id_restaurante)  
14    REFERENCES Restaurante(id_restaurante)  
15 );  
16  
17  
18 CREATE TABLE Pedido (  
19   id_pedido INT AUTO_INCREMENT PRIMARY KEY,  
20   id_restaurante INT,  
21   data_pedido DATE NOT NULL,  
22   status VARCHAR(20) NOT NULL,  
23   FOREIGN KEY (id_restaurante)  
24     REFERENCES Restaurante(id_restaurante)  
25 );
```

STDIN
Input for the program (Optional)

Output:

Field	Type	Null	Key	Default	Extra
id_restaurante	int	NO	PRI	NULL	auto_increment
nome	varchar(100)	NO		NULL	
endereco	varchar(200)	NO		NULL	

Field	Type	Null	Key	Default	Extra
id_menu	int	NO	PRI	NULL	auto_increment
id_restaurante	int	YES	MUL	NULL	
descricao	varchar(150)	NO		NULL	
disponibilidade	tinyint(1)	NO		1	
preco	decimal(10,2)	YES		NULL	

OneCompilerUpgrade

AI MySQL Run

Save Login

queries.sql

```
1 CREATE TABLE Restaurante (  
2   id_restaurante INT AUTO_INCREMENT PRIMARY KEY,  
3   nome VARCHAR(100) NOT NULL,  
4   endereco VARCHAR(200) NOT NULL,  
5   telefone VARCHAR(15)  
6 );  
7  
8 CREATE TABLE Menu (  
9   id_menu INT AUTO_INCREMENT PRIMARY KEY,  
10  id_restaurante INT,  
11  descricao VARCHAR(150) NOT NULL,  
12  disponibilidade BOOLEAN NOT NULL,  
13  FOREIGN KEY (id_restaurante)  
14    REFERENCES Restaurante(id_restaurante)  
15 );  
16  
17  
18 CREATE TABLE Pedido (  
19   id_pedido INT AUTO_INCREMENT PRIMARY KEY,  
20   id_restaurante INT,  
21   data_pedido DATE NOT NULL,  
22   status VARCHAR(20) NOT NULL,  
23   FOREIGN KEY (id_restaurante)  
24     REFERENCES Restaurante(id_restaurante)  
25 );
```

STDIN
Input for the program (Optional)

Output:

id_menu	int	NO	PRI	NULL	auto_increment
id_restaurante	int	YES	MUL	NULL	
descricao	varchar(150)	NO		NULL	
disponibilidade	tinyint(1)	NO		1	
preco	decimal(10,2)	YES		NULL	

Field	Type	Null	Key	Default	Extra
id_pedido	int	NO	PRI	NULL	auto_increment
id_restaurante	int	YES	MUL	NULL	
data_pedido	date	NO		NULL	
status	varchar(20)	NO		NULL	
id_menu	int	YES	MUL	NULL	
quantidade	int	NO		1	

Descrição das alterações

Na tabela **Menu**, foi adicionada a coluna **preco** para armazenar o preço dos itens. A coluna **disponibilidade** foi modificada para ter o valor padrão **TRUE**.

Na tabela **Pedido**, foram adicionadas as colunas **id_menu** e **quantidade**. A coluna **id_menu** permite identificar o item do menu relacionado ao pedido, enquanto **quantidade** registra a quantidade solicitada.

Na tabela **Restaurante**, a coluna **telefone** foi removida porque foi considerada desnecessária.

Perguntas de Reflexão

1. Por que é importante planejar as modificações de estrutura em um banco de dados em produção?

É importante planejar as modificações para evitar perda de dados, erros e problemas de funcionamento no sistema. Alterações mal planejadas podem afetar os usuários e outras aplicações que utilizam o banco.

2. Qual é a vantagem de definir valores padrão, como no caso da coluna **disponibilidade**?

Os valores padrão facilitam o cadastro dos dados e evitam que informações importantes fiquem sem valor. No caso da coluna **disponibilidade**, o valor padrão **TRUE** faz com que o item seja considerado disponível quando nenhum valor for informado.

Aula 03 - Manipulação de dados em um banco de dados de comércio eletrônico com comandos DML

queries.sql

67

-- 4. Inserir itens nos pedidos

68 INSERT INTO Itens_Pedido (id_pedido, id_produto, quantidade)

69 VALUES

70 (1, 1, 2),

71 (1, 3, 1),

72 (2, 2, 1),

73 (2, 3, 1);

74

75

76

-- Conferir os itens

77 SELECT * FROM Itens_Pedido;

78

79

80

81

-- 1. Atualizar preço dos fones

82 UPDATE Produtos

83 SET preco = 169.90

84 WHERE nome = 'Fones de Ouvido Bluetooth';

85

86

87

-- 2. Atualizar endereço do João

88 UPDATE Clientes

89 SET endereco = 'Rua C, 789'

90 WHERE nome = 'João Silva';

91

--

I/O

AI Agent

146 ms

STDIN

Input for the program (Optional)

Output:

id_cliente	nome	email	endereco
1	João Silva	joao@email.com	Rua A, 123
2	Maria Oliveira	maria@email.com	Avenida B, 456

id_pedido	id_cliente	data_pedido	status
1	1	2024-11-01	Em Processamento
2	2	2024-11-02	Em Processamento

id_item	id_pedido	id_produto	quantidade
1	1	1	2
2	1	3	1
3	2	2	1
4	2	3	1

queries.sql

67

-- 4. Inserir itens nos pedidos

68 INSERT INTO Itens_Pedido (id_pedido, id_produto, quantidade)

69 VALUES

70 (1, 1, 2),

71 (1, 3, 1),

72 (2, 2, 1),

73 (2, 3, 1);

74

75

76

-- Conferir os itens

77 SELECT * FROM Itens_Pedido;

78

79

80

81

-- 1. Atualizar preço dos fones

82 UPDATE Produtos

83 SET preco = 169.90

84 WHERE nome = 'Fones de Ouvido Bluetooth';

85

86

87

-- 2. Atualizar endereço do João

88 UPDATE Clientes

89 SET endereco = 'Rua C, 789'

90 WHERE nome = 'João Silva';

91

I/O

AI Agent

146 ms

STDIN

Input for the program (Optional)

Output:

id_produto	nome	tipo	preco
1	Camiseta Algodão	Físico	59.90
2	Curso Python On-line	Digital	199.90
3	Fones de Ouvido Bluetooth	Físico	169.90

id_cliente	nome	email	endereco
1	João Silva	joao@email.com	Rua C, 789
2	Maria Oliveira	maria@email.com	Avenida B, 456

queries.sql

89 UPDATE Clientes

90 SET endereco = 'Rua C, 789'

91 WHERE nome = 'João Silva';

92

93

94 -- 3. Alterar status do pedido da Maria

95 UPDATE Pedidos

96 SET status = 'Cancelado'

97 WHERE id_pedido = 2;

98

99

100

-- Conferir as alterações

101 SELECT * FROM Produtos;

102 SELECT * FROM Clientes;

103 SELECT * FROM Pedidos;

104

105

106

-- 1. Excluir o Curso Python do Pedido 2

107 DELETE FROM Itens_Pedido

108 WHERE id_pedido = 2

109 AND id_produto = 2;

110

111

112

-- 2. Verificar o pedido do João

113

I/O

AI Agent

146 ms

STDIN

Input for the program (Optional)

Output:

id_pedido	id_cliente	data_pedido	status
1	1	2024-11-01	Em Processamento
2	2	2024-11-02	Cancelado

id_pedido	id_cliente	data_pedido	status
1	1	2024-11-01	Em Processamento

id_produto	nome	tipo	preco
1	Camiseta Algodão	Físico	59.90
2	Curso Python On-line	Digital	199.90

OneCompiler

Upgrade

AI

MySQL

Run

Save

Login

queries.sql

125 WHERE id_pedido = 1;

126

127

128 -- Excluir o cliente João

129 DELETE FROM Clientes

130 WHERE id_cliente = 1;

131

132

133

134 -- 1. Todos os produtos

135 SELECT * FROM Produtos;

136

137

138 -- 2. Pedidos da Maria Oliveira

139 SELECT Pedidos.*

140 FROM Pedidos

141 JOIN Clientes

142 ON Pedidos.id_cliente = Clientes.id_cliente

143 WHERE Clientes.nome = 'Maria Oliveira';

144

145

146 -- 3. Todos os itens do Pedido 1

147 SELECT *

148 FROM Itens_Pedido

149 WHERE id_pedido = 1;

I/O

AI Agent

146 ms

STDIN

Input for the program (Optional)

Output:

id_pedido	id_cliente	data_pedido	status
1	1	2024-11-01	Em Processamento

id_produto	nome	tipo	preco
1	Camiseta Algodão	Físico	59.90
2	Curso Python On-line	Digital	199.90
3	Fones de Ouvido Bluetooth	Físico	169.90

id_pedido	id_cliente	data_pedido	status
2	2	2024-11-02	Cancelado

Perguntas de Reflexão

1. Por que é importante utilizar UPDATE com cuidado em um banco de dados?

É importante utilizar o UPDATE com cuidado porque ele pode alterar vários registros. A cláusula WHERE deve ser utilizada para garantir que somente os dados desejados sejam modificados.

2. Quais cuidados devem ser tomados ao excluir dados de clientes ou pedidos?

É necessário verificar as chaves estrangeiras e os dados relacionados antes de excluir um registro. Isso evita problemas de integridade e a perda indevida de informações relacionadas.

Aula 04 - Implementando transações para garantir a integridade dos dados em operações DML

Código com erro:

OneCompilerUpgrade

AI MySQLRun

SaveLogin

queries.sql

```
1 -- Apagar tabelas caso já existam
2 DROP TABLE IF EXISTS Itens_Pedido;
3 DROP TABLE IF EXISTS Pedidos;
4 DROP TABLE IF EXISTS Clientes;
5
6
7
8 CREATE TABLE Clientes (
9     id_cliente INT AUTO_INCREMENT PR
10     nome VARCHAR(100),
11     email VARCHAR(100),
12     endereco VARCHAR(255)
13 );
14
15 CREATE TABLE Pedidos (
16     id_pedido INT AUTO_INCREMENT PRI
17     id_cliente INT,
18     data_pedido DATE,
19     status VARCHAR(20),
20     FOREIGN KEY (id_cliente)
21     REFERENCES Clientes(id_clien
22 );
23
24 CREATE TABLE Itens_Pedido (
```

STDIN

Input for the program (Optional)

Output

id_cliente	nome	email	endereco
1	Carlos Mendes	carlos@email.com	Rua das Flores, 789

id_pedido	id_cliente	data_pedido	status
1	1	2024-11-10	Em Processamento

id_item	id_pedido	id_produto	quantidade	preco
1	1	101	2	299.90

ERROR 1452 (23000) at line 134: Cannot add or update a child row: a foreign key constraint fails (`sandbox db`.`ite

OneCompilerUpgrade

AI MySQLRun

Save

queries.sql

```
1 DROP TABLE IF EXISTS Itens_Pedido;
2 DROP TABLE IF EXISTS Pedidos;
3 DROP TABLE IF EXISTS Clientes;
4
5 CREATE TABLE Clientes (
6     id_cliente INT AUTO_INCREMENT PRIMARY KEY,
7     nome VARCHAR(100),
8     email VARCHAR(100),
9     endereco VARCHAR(255)
10 );
11
12 CREATE TABLE Pedidos (
13     id_pedido INT AUTO_INCREMENT PRIMARY KEY,
14     id_cliente INT,
15     data_pedido DATE,
16     status VARCHAR(20),
17     FOREIGN KEY (id_cliente)
18     REFERENCES Clientes(id_cliente)
19 );
20
21 CREATE TABLE Itens_Pedido (
22     id_item INT AUTO_INCREMENT PRIMARY KEY,
23     id_pedido INT,
24     id_produto INT,
25     quantidade INT,
```

STDIN

Input for the program (Optional)

Output

id_cliente	nome	email	endereco
1	Carlos Mendes	carlos@email.com	Rua das Flores, 789

id_pedido	id_cliente	data_pedido	status
1	1	2024-11-10	Em Processamento

id_item	id_pedido	id_produto	quantidade	preco
1	1	101	2	299.90

OneCompiler

Upgrade

AI

MySQL

Run

Save

Login

queries.sql

```
1 DROP TABLE IF EXISTS Itens_Pedido;
2 DROP TABLE IF EXISTS Pedidos;
3 DROP TABLE IF EXISTS Clientes;
4
5 CREATE TABLE Clientes (
6     id_cliente INT AUTO_INCREMENT PRIMARY KEY,
7     nome VARCHAR(100),
8     email VARCHAR(100),
9     endereco VARCHAR(255)
10 );
11
12 CREATE TABLE Pedidos (
13     id_pedido INT AUTO_INCREMENT PRIMARY KEY,
14     id_cliente INT,
15     data_pedido DATE,
16     status VARCHAR(20),
17     FOREIGN KEY (id_cliente)
18     REFERENCES Clientes(id_cliente)
19 );
20
21 CREATE TABLE Itens_Pedido (
22     id_item INT AUTO_INCREMENT PRIMARY KEY,
23     id_pedido INT,
24     id_produto INT,
25     quantidade INT;
```

I/O

AI Agent

191 ms

STDIN

Input for the program (Optional)

Output:

id_cliente	nome	email	endereco
1	Carlos Mendes	carlos@email.com	Rua das Flores, 789
2	Carlos Mendes	carlos@email.com	Rua das Flores, 789

id_pedido	id_cliente	data_pedido	status
1	1	2024-11-10	Em Processamento
2	2	2024-11-10	Em Processamento

id_item	id_pedido	id_produto	quantidade	preco
1	1	101	2	299.90
2	2	101	2	299.90

OneCompiler

Upgrade

AI

MySQL

Run

Save

Login

queries.sql

```
1 DROP TABLE IF EXISTS Itens_Pedido;
2 DROP TABLE IF EXISTS Pedidos;
3 DROP TABLE IF EXISTS Clientes;
4
5 CREATE TABLE Clientes (
6     id_cliente INT AUTO_INCREMENT PRIMARY KEY,
7     nome VARCHAR(100),
8     email VARCHAR(100),
9     endereco VARCHAR(255)
10 );
11
12 CREATE TABLE Pedidos (
13     id_pedido INT AUTO_INCREMENT PRIMARY KEY,
14     id_cliente INT,
15     data_pedido DATE,
16     status VARCHAR(20),
17     FOREIGN KEY (id_cliente)
18     REFERENCES Clientes(id_cliente)
19 );
20
21 CREATE TABLE Itens_Pedido (
22     id_item INT AUTO_INCREMENT PRIMARY KEY,
23     id_pedido INT,
24     id_produto INT,
25     quantidade INT;
```

I/O

AI Agent

191 ms

STDIN

Input for the program (Optional)

Output:

id_cliente	nome	email	endereco
1	Carlos Mendes	carlos@email.com	Rua das Flores, 789
2	Carlos Mendes	carlos@email.com	Rua das Flores, 789

id_pedido	id_cliente	data_pedido	status
1	1	2024-11-10	Em Processamento
2	2	2024-11-10	Em Processamento

id_item	id_pedido	id_produto	quantidade	preco
1	1	101	2	299.90
2	2	101	2	299.90

OneCompiler

Upgrade

AI

MySQL

Run

Save

Login

queries.sql

```
1 DROP TABLE IF EXISTS Itens_Pedido;
2 DROP TABLE IF EXISTS Pedidos;
3 DROP TABLE IF EXISTS Clientes;
4
5 CREATE TABLE Clientes (
6     id_cliente INT AUTO_INCREMENT PRIMARY KEY,
7     nome VARCHAR(100),
8     email VARCHAR(100),
9     endereco VARCHAR(255)
10 );
11
12 CREATE TABLE Pedidos (
13     id_pedido INT AUTO_INCREMENT PRIMARY KEY,
14     id_cliente INT,
15     data_pedido DATE,
16     status VARCHAR(20),
17     FOREIGN KEY (id_cliente)
18     REFERENCES Clientes(id_cliente)
19 );
20
21 CREATE TABLE Itens_Pedido (
22     id_item INT AUTO_INCREMENT PRIMARY KEY,
23     id_pedido INT,
24     id_produto INT,
25     quantidade INT;
```

I/O

AI Agent

191 ms

STDIN

Input for the program (Optional)

Output:

id_cliente	nome	email	endereco
1	Carlos Mendes	carlos@email.com	Rua das Flores, 789
2	Carlos Mendes	carlos@email.com	Rua das Flores, 789

id_pedido	id_cliente	data_pedido	status
1	1	2024-11-10	Em Processamento
2	2	2024-11-10	Em Processamento

id_item	id_pedido	id_produto	quantidade	preco
1	1	101	2	299.90
2	2	101	2	299.90

Descrição:

Eu troquei algumas partes do código porque ele estava apresentando erros no OneCompiler. Fiz as alterações para deixar o código mais simples e compatível com a ferramenta. Também organizei os IDs das tabelas para conseguir fazer a transação corretamente e testar o COMMIT e o ROLLBACK.

Considerações Finais - Perguntas

1. Como vocês podem garantir que a transação será revertida se houver um erro ao inserir dados na tabela Itens_Pedido?

Resposta: As operações devem ser realizadas dentro de uma transação iniciada com START TRANSACTION ou BEGIN. Se ocorrer algum erro, deve-se utilizar o comando ROLLBACK para desfazer as alterações realizadas durante a transação. Se tudo ocorrer corretamente, utiliza-se COMMIT para confirmar os dados.

2. Como a cláusula ROLLBACK pode ser usada para reverter transações parciais em caso de falha em qualquer etapa do processo?

Resposta: O ROLLBACK desfaz as alterações realizadas desde o início da transação que ainda não foram confirmadas. Assim, se uma etapa apresentar erro, as operações anteriores também podem ser revertidas, evitando que apenas parte dos dados seja armazenada no banco.

Conclusão

As transações garantem maior segurança e integridade aos dados, pois permitem confirmar todas as operações com COMMIT ou desfazer as alterações com ROLLBACK caso ocorra algum problema.